

Contents

map-salesforce	1
Project configuration	1
Project layout	2
Packaging model	2
Authentication	3
Aliases explained	3
Dev Hub basics	3
Recommended commands	4
Plugins	4
Good beginner habits	4
Notes	5

map-salesforce

This repository is a Salesforce DX project for the **map-salesforce** workspace, using **sf/sfdx** project configuration, sandbox-first defaults, and a two-package layout consisting of a base package and a dependent feature package.

It is also set up so people who are newer to Salesforce CLI can understand the basics of what the CLI is, how org login works, and how this repository uses shared TDS org aliases plus personal Dev Hub aliases.

CLI Basics

Salesforce (SF) Command Line Interface (CLI) is a free SF command-line tool that lets people work with SF orgs from a terminal or command prompt instead of doing every task through the browser UI.

A command-line interface, or CLI, is simply a text-based tool where tasks are done by typing commands.

For this project, SF CLI is mainly used to:

- Log into the TDS Salesforce org.
- Set a default org and Dev Hub alias for local development.
- Work with metadata in this repository.
- Support scratch-org and packaging workflows.

Project configuration

The current project configuration targets API version **63.0**, uses **https://test.salesforce.com** as the project login default, and sets **oauthLocalPort** to **1718** to avoid common localhost callback conflicts during browser-based auth.

The project-level **sfdcLoginUrl** is appropriate for sandbox-first workflows, but explicit CLI login commands can override it with a My Domain URL such as **https://tdstelecom.my.salesforce.com** when needed.

The `plugins` block enables `salesforcedx-vscode-core.enable-sobject-refresh-on-startup`, which helps keep local SObject definitions fresh for Salesforce VS Code workflows.

The `replacements` block defines environment substitutions for custom metadata and named credentials, replacing `API_BASE_URL_PLACEHOLDER` with `https://sandbox.my.salesforce.com` and `ENV_NAME_PLACEHOLDER` with `SANDBOX`.

The project also enables these `sourceBehaviorOptions`:

- `decomposeCustomLabelsBeta2`
- `decomposeExternalServiceRegistrationBeta`
- `decomposePermissionSetBeta2`
- `decomposeSharingRulesBeta`
- `decomposeWorkflowBeta`

Project layout

The repository defines three package directories: `force-app` as the default base package root, `packages/feature` as a second package that depends on `map-base`, and `unpackaged/config` for unpackaged metadata or environment-specific configuration.

Path	Purpose	Notes
<code>force-app</code>	Base package source	Default package directory, mapped to <code>map-base</code> .
<code>packages/feature</code>	Feature package source	Depends on <code>map-base@1.0.0.LATEST</code> .
<code>unpackaged/config</code>	Unpackaged metadata	Not marked as default and not tied to a package alias.

Packaging model

`packageAliases` maps readable package names to Salesforce package IDs and package version IDs so packaging commands can use aliases instead of raw IDs.

The current package model is:

Alias	Type	Value
<code>map-base</code>	Package ID	<code>0Ho0000000000001AAA</code>
<code>map-base@1.0.0-1</code>	Package Version ID	<code>04t0000000000001AAA</code>
<code>map-feature</code>	Package ID	<code>0Ho0000000000002AAA</code>
<code>map-feature@1.0.0-1</code>	Package Version ID	<code>04t0000000000002AAA</code>

The feature package depends on `map-base` version `1.0.0.LATEST`, so package creation and install workflows should account for base-first ordering even though `pushPackageDirectoriesSequentially` is currently `false`.

Authentication

The easiest login method for most users is browser-based login with `sf org login web`, which opens a browser window and lets the user sign in with normal Salesforce credentials.

For TDS work, use the My Domain URL directly when authorizing the CLI:

```
sf org login web \
  --instance-url https://tdstelecom.my.salesforce.com \
  --alias tds \
  --set-default
```

If the same org is also being used as a Dev Hub, authorize it like this:

```
sf org login web \
  --instance-url https://tdstelecom.my.salesforce.com \
  --alias map-dev \
  --set-default-dev-hub
```

Salesforce recommends using `--alias` for readable org names and `--set-default-dev-hub` when authorizing a Dev Hub org for scratch-org creation and packaging workflows.

Aliases explained

An alias is just a friendly nickname for a Salesforce org in the CLI, such as `tds`, `devhub`, or `map-dev`.

This repository uses a simple pattern:

- `tds` is the shared standard alias for the TDS work org.
- A personal alias such as `map-dev` can be used for the same org when that org is acting as the developer's default Dev Hub.

That makes commands easier to read and helps teams stay consistent.

Dev Hub basics

A Dev Hub is a Salesforce org with Dev Hub enabled so it can create and manage scratch orgs for development workflows.

Because the TDS org is being used as a Dev Hub, each developer can give that same org a personal Dev Hub alias and then use it for scratch-org commands.

Recommended commands

Common commands:

```
# show CLI version  
sf --version
```

```
# log into the TDS org
```

```
sf org login web --instance-url https://tdstelecom.my.salesforce.com --alias tds --set-default-org
```

```
# log into the same org as a Dev Hub
```

```
sf org login web --instance-url https://tdstelecom.my.salesforce.com --alias map-dev --set-default-org
```

```
# list orgs and aliases
```

```
sf org list
```

```
sf alias list
```

```
# open the default org in a browser
```

```
sf org open
```

```
# create a scratch org
```

```
sf org create scratch --definition-file config/project-scratch-def.json --alias map-scratch
```

```
# inspect plugins
```

```
sf plugins --core
```

```
sf plugins
```

These commands are enough for most people to confirm the CLI is installed, the org login worked, and the local environment is ready.

Plugins

Salesforce CLI supports plugins, which add groups of commands to the CLI.

Many important capabilities already come from core or just-in-time plugins, but this repository's setup script explicitly installs additional official plugins such as `@salesforce/plugin-packaging` and `@salesforce/plugin-devops-center` when needed.

Good beginner habits

- Use aliases so org commands are easier to remember.
- Use browser login first because it is the simplest path for most users, especially with MFA or SSO.
- Start by learning a few commands: `sf --version`, `sf org list`, `sf org open`, and `sf plugins`.
- Use the repo's standard TDS alias pattern so the team stays consistent.

Notes

- `sf` still uses `sfdx-project.json` or `sfdx-project.jsonc` as the project configuration format; there is no separate `sf-project.json` format.
- `oauthLocalPort: 1718` is useful when default localhost callback ports are unavailable during browser-based auth.
- `sfdcLoginUrl` sets a project default, but `--instance-url` on the command line takes precedence for explicit org login commands.